

Design Coffee Vending Machine

Interview frame: 60 minutes · junior candidate · Java 25

Understanding the Problem

A coffee vending machine offers a small menu, collects money, checks whether the selected drink can be prepared, consumes its recipe, and returns change. The interesting part of the interview is the order of those steps. An unsuccessful purchase must leave both the customer's money and the ingredient stock in a sensible state.

Prompt: Design a coffee vending machine that lets a customer select a drink, insert coins, dispense it, receive change, or cancel the purchase.

Clarifying Questions

Candidate: Which drinks and ingredients should the machine support?

Interviewer: Configure the menu when the machine starts. For this exercise, recipes use water in millilitres, coffee in grams, and milk in millilitres.

The menu should contain data rather than recipe-specific branches. `Ingredient` can be an enum because the set is closed for this round. Each recipe maps an ingredient to the required integer amount.

Candidate: Are drink customizations such as extra shots or sugar required?

Interviewer: No. Each menu entry has one fixed recipe and price.

One `Beverage` record can carry the code, display name, price, and recipe. The demo creates fixed recipes with `Map.of`.

Candidate: How does payment work?

Interviewer: Accept coins worth 25, 50, 100, or 200 cents. Prices and balances use integer cents. Assume the machine can always return the calculated change.

This removes floating-point rounding and physical change-coin inventory from the base design. The completed result only needs the amount of change.

Candidate: Can money be inserted before a drink is selected?

Interviewer: Keep one simple order: select, insert coins, then dispense. The customer may cancel at any point before dispensing.

The machine needs one active selection and one balance. It rejects a second selection until the current session finishes or is cancelled.

Candidate: Is brewing asynchronous, and can several customers use the same instance at once?

Interviewer: No. Treat preparation as a synchronous simulation for one customer.

We can return the completed drink from `dispense()`. Motor control, brewing progress, timers, and locks would distract from the junior-level object model.

Final Requirements

- ¹ Use the menu configured at startup to look up drinks by code.

- 2 Store a fixed price and ingredient recipe for each beverage.
- 3 Select one known beverage only when its recipe is currently available.
- 4 Accept only 25, 50, 100, and 200 cent coins after a selection.
- 5 Dispense after the inserted balance covers the selected price.
- 6 Consume every recipe ingredient exactly once on a successful dispense.
- 7 Return the beverage, total paid, and calculated change, then clear the session.
- 8 Cancel a purchase, return the inserted balance, and consume no ingredients.
- 9 Reject invalid operation order, unsupported coins, insufficient payment, or insufficient ingredients without partial ingredient consumption.

Out of Scope

- heaters, grinders, pumps, sensors, and physical failure handling;
- card authorization and payment settlement;
- tracking the denominations available for change;
- custom drink sizes, extra shots, sugar, or temperature choices;
- administrator authentication and restocking operations;
- persistence, networking, telemetry, and concurrent customers.

Setup Assumptions

The demo supplies a valid menu with unique codes, positive prices and recipe amounts, and non-negative starting stock. Recipes are fixed `Map` of values. These are setup assumptions, so constructors simply store the data they receive.

Customer actions still need checks: a code may not be on the menu, a coin may be unsupported, or the customer may try to dispense too early. Ingredient stock can also be insufficient. Those cases affect the purchase workflow and belong in the implementation.

Finding the Core Entities

Some familiar coffee-machine concepts do not earn classes in this contract. A grinder and brewer matter to physical hardware, but the simulation has one synchronous `dispense()` operation. A customer supplies commands and has no stored identity. Coins have no behavior beyond membership in a fixed set of accepted integer values.

Candidate	Keep as	Reason
Coffee vending machine	Class	Coordinates the menu, customer session, payment checks, and preparation.
Ingredient inventory	Class	Owns mutable quantities and the all-or-nothing consumption rule.
Beverage	Record	Carries fixed menu data and a recipe.
Dispense result	Immutable record	Preserves the completed purchase after the machine resets.
Ingredient	Enum	Defines the closed ingredient keys and embeds measurement units in their names.

Candidate	Keep as	Reason
Coin	Integer plus a set	Four fixed values need validation, not an object lifecycle.
Customer	Reject	Identity does not affect any requirement.
Brewer and grinder	Reject for the base	Hardware execution is outside the synchronous simulation.

Responsibilities at a Glance

Type	Responsibility	State or invariant owned
CoffeeVendingMachine	Public purchase workflow	At most one selected beverage and its balance.
IngredientInventory	Check and consume recipes	Quantities stay non-negative; failed consumption changes nothing.
Beverage	Describe one menu option	Trusted code, name, price, and fixed recipe.
DispenseResult	Report one completed purchase	Values computed by the successful dispense.
Ingredient	Give ingredient quantities semantic keys	Each enum name includes its measurement unit.

Exploring the Design

Decision: where should recipe consumption live?

Consider a latte that needs 40 ml of water, 18 g of coffee, and 120 ml of milk. The inventory currently has exactly enough water and coffee but only 100 ml of milk.

Bad: decrement ingredients while checking them

A direct implementation inside `dispense()` may look compact:

```
waterMl -= 40
coffeeGrams -= 18
if milkMl < 120: reject
milkMl -= 120
```

The rejection arrives too late. Water and coffee have already been removed even though no latte was produced. The operation broke the requirement that failure preserve the inventory.

Good: validate the full recipe first

The machine can make two passes: first check every amount, then perform every decrement.

```
require waterMl >= 40
require coffeeGrams >= 18
require milkMl >= 120

waterMl -= 40
coffeeGrams -= 18
milkMl -= 120
```

This is correct for one drink, but each new recipe adds another branch to the purchase orchestrator. The class that manages payment would also know every ingredient field.

Great for this scope: let the inventory consume a recipe

Represent a recipe as `Map<Ingredient, Integer>`. `IngredientInventory.consume(recipe)` checks that stock covers every recipe entry in one pass and mutates quantities in a second pass. The same operation prepares espresso or latte without changing the machine workflow.

The map adds a small amount of indirection, but it keeps one stock owner and one mutation path. A junior candidate can explain and code both loops within the interview.

Recommendation: Implement in interview. Keep the generic two-pass consumption method. Mention if asked: a richer unit type could prevent mixing grams and millilitres in a larger model.

Decision: should the machine store an explicit state enum?

An enum with `IDLE`, `SELECTED`, and `READY` seems natural. `READY`, however, depends on the selected beverage's price and the current balance. Every accepted coin could require an extra state update.

The selected beverage and balance already contain the needed facts:

- no selected beverage means the machine is idle;
- a selected beverage means coins may be inserted;
- `balanceCents >= beverage.priceCents()` means dispensing has enough money.

Deriving these conditions avoids a second mutable representation. Each public method still checks whether its transition is legal.

Recommendation: Implement in interview. Use the two fields. Production extension: introduce explicit brewing states when hardware reports asynchronous progress or failure.

Class Design

CoffeeVendingMachine: purchase orchestrator

`CoffeeVendingMachine` is the only public workflow entry point. It resolves menu codes, enforces operation order, tracks the current payment, and asks the inventory to consume the selected recipe. It does not know how individual ingredient quantities are stored or changed.

Requirement or invariant	State needed	Why this owner
Resolve a configured drink	<code>menuByCode</code>	The machine owns its selectable menu boundary.
Allow one active purchase	selected beverage	Retains the menu object resolved at selection.
Accumulate inserted money	<code>balanceCents</code>	Balance spans coin insertions, cancellation, and dispensing.
Validate denominations	<code>SUPPORTED_COINS</code>	Coin acceptance belongs at the public input boundary.
Prepare the selected recipe	inventory collaborator	Ingredient mutation belongs to its dedicated owner.

Caller need or transition	Method	Result or mutation
Start a purchase	<code>select(code)</code>	Stores one known, available beverage.
Add payment	<code>insertCoin(cents)</code>	Validates and increases the active balance.
Finish a purchase	<code>dispense()</code>	Consumes one recipe, resets the session, and returns the result.

Caller need or transition	Method	Result or mutation
Abandon a purchase	<code>cancel()</code>	Clears the session and returns the balance.

```
class CoffeeVendingMachine
  menuByCode: Map<String, Beverage>
  inventory: IngredientInventory
  selected: Beverage?
  balanceCents: int

  select(code): void
  insertCoin(cents): void
  dispense(): DispenseResult
  cancel(): int
```

Invariant: A completed dispense consumes one whole recipe and clears the session. A rejected operation cannot partially consume a recipe.

Knowledge boundary: The machine knows recipe requirements through `Beverage`, but it does not access or decrement ingredient fields itself.

Collaborators: It reads `Beverage` menu data, delegates stock checks and consumption to `IngredientInventory`, and creates `DispenseResult`.

Keeping the selected object makes its price and recipe available directly. The menu stays fixed during a purchase, so storing only the code would add a lookup with no useful behavior. The private `clearSession()` helper resets the two fields used by both cancellation and dispensing.

IngredientInventory: ingredient stock owner

`IngredientInventory` stores the physical ingredient quantities. It is the only class allowed to decrement them. Each operation visits the recipe entries; an ingredient absent from stock has quantity zero. The constructor copies the starting quantities into its own map so the inventory can mutate them.

Requirement or invariant	State needed	Why this owner
Track all current quantities	<code>quantities</code> enum map	One map is the ingredient source of truth.
Decide recipe availability	recipe input plus quantities	The answer depends only on inventory data.
Prevent partial consumption	two-pass operation	All amounts are checked before the first decrement.

Caller need or transition	Method	Result or mutation
Check menu availability	<code>canPrepare(recipe)</code>	Returns whether every required amount is present.
Prepare one drink	<code>consume(recipe)</code>	Rejects without mutation or decrements the complete recipe.
Inspect deterministic state	<code>quantityOf(ingredient)</code>	Supports the demo and correctness tests.

```
class IngredientInventory
  quantities: Map<Ingredient, Integer>

  canPrepare(recipe): boolean
  consume(recipe): void
  quantityOf(ingredient): int
```

Invariant: Starting from the valid setup, quantities stay non-negative. If one recipe requirement cannot be met, all quantities retain their previous values. The order of the map entries does not affect this guarantee.

Knowledge boundary: The inventory understands ingredient amounts, not menu codes, prices, coins, or customer-session order.

Collaborators: It receives immutable recipe maps from a `Beverage`; only `CoffeeVendingMachine` calls it during a purchase.

Beverage: fixed menu entry

Beverage represents the stable information needed after a code is selected. A Java record supplies its constructor and accessors. The recipe is already immutable because the demo uses `Map.of`; the record adds no validation or copying.

Requirement or invariant	State needed	Why this owner
Select by a compact value	<code>code</code>	The code identifies the menu entry.
Display the chosen drink	<code>name</code>	The completed result needs a readable name.
Check payment	<code>priceCents</code>	Price belongs to the menu entry and uses an explicit unit.
Consume ingredients	<code>recipe</code>	The fixed recipe defines the beverage.

Caller need or transition	Method	Result or mutation
Configure a drink	record constructor	Stores the supplied demo values.
Read menu data	record accessors	Exposes the fixed fields and recipe.

```
record Beverage
  code: String
  name: String
  priceCents: int
  recipe: Map<Ingredient, Integer>
```

Setup contract: Prices and recipe amounts are positive, and the demo passes immutable recipe maps. A record alone does not make a mutable map immutable; this property comes from how the demo constructs it.

Knowledge boundary: A beverage does not know inventory quantities, payment state, or preparation order.

Collaborators: The machine indexes it; the inventory reads its recipe.

DispenseResult and Ingredient: supporting values

`DispenseResult` is an immutable snapshot of a successful purchase. The machine clears its session before accepting another customer, so returning a record keeps the finished values available to the caller. `Ingredient` supplies semantic recipe keys. Its constants include units, such as `WATER_ML` and `COFFEE_GRAMS`, which prevents an unlabeled amount from crossing the API.

Requirement or invariant	State needed	Why this owner
Identify the prepared drink	<code>beverage</code>	The result describes the completed action.
Report payment and change	<code>paidCents</code> , <code>changeCents</code>	These values remain useful after session reset.
Distinguish recipe units	enum constants	The key carries the measurement meaning.

Caller need or transition	Method	Result or mutation
Receive purchase outcome	record accessors	Exposes immutable data with no follow-up mutation.
Preserve a completed result	record constructor	Stores the outcome already computed by <code>dispense()</code> .
Address an ingredient	enum value	Supplies a closed, type-safe map key.

```

record DispenseResult
  beverage: Beverage
  paidCents: int
  changeCents: int

enum Ingredient
  WATER_ML
  COFFEE_GRAMS
  MILK_ML

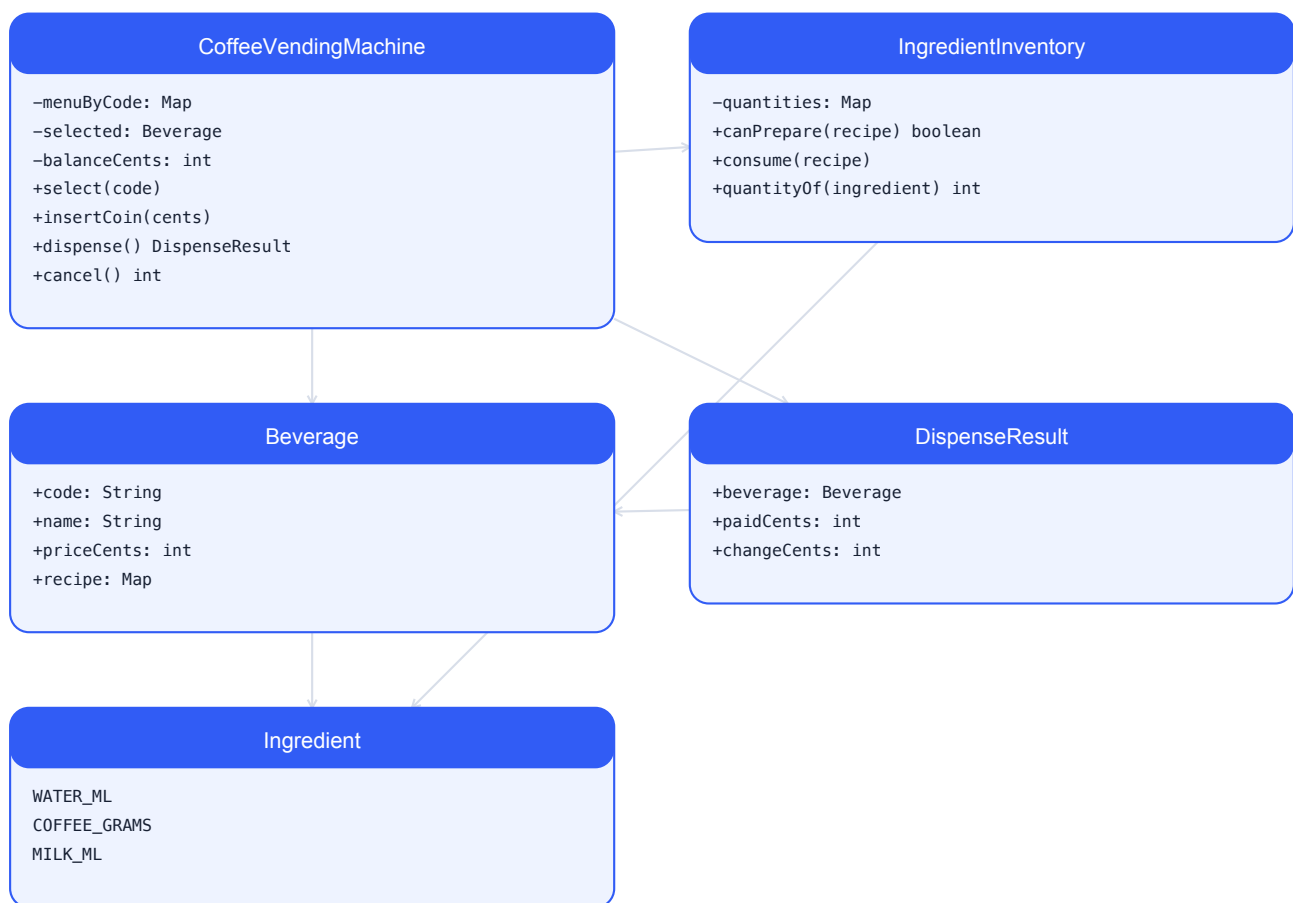
```

Invariant owner: `dispense()` computes `changeCents` as `paidCents - beverage.priceCents()` after checking payment. The result records that calculation. Ingredient keys state their unit.

Knowledge boundary: These values do not coordinate the purchase or mutate ingredient quantities.

Collaborators: `CoffeeVendingMachine` creates the result. `Beverage` and `IngredientInventory` share the ingredient keys.

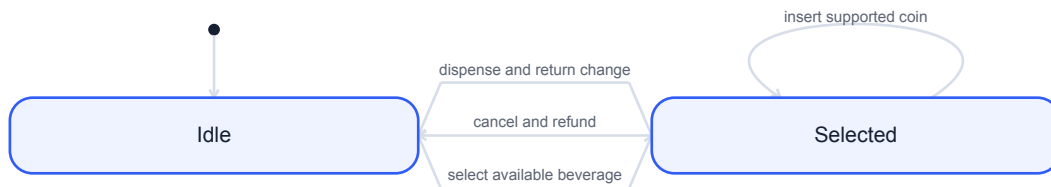
Final Class Design



Relationships

CoffeeVendingMachine → Beverage: owns menu
 CoffeeVendingMachine → IngredientInventory: owns workflow access
 CoffeeVendingMachine → DispenseResult: returns
 Beverage → Ingredient: recipe keys
 IngredientInventory → Ingredient: quantity keys
 DispenseResult → Beverage: prepared drink

The machine owns the customer session. The inventory owns all mutable ingredient stock. `Beverage` connects them through an immutable recipe, so payment logic never performs ingredient-specific decrements.



The lifecycle is intentionally small. Payment readiness is calculated from the selected price and current balance rather than stored as another state.

Core Implementation

Start with `select`, `IngredientInventory.consume`, and `dispense`, since they establish lookup, stock ownership, and purchase completion. Then add `insertCoin`, `cancel`, the small data types, and the demo. All of this belongs to the interview implementation.

Operation: select an available beverage

`select (code)` first protects the one-session rule. It then resolves the code and asks the inventory whether the fixed recipe is available. `selected` changes only after all checks pass.

```

if a selected beverage already exists: reject
beverage = menuByCode[code]
if beverage is missing: reject
if inventory cannot prepare beverage.recipe: reject
selected = beverage
  
```

An unavailable latte does not create a half-started session. The customer may immediately select another menu item.

The null check belongs immediately after `menuByCode.get (code)`: that lookup can fail for a caller's choice. Once a beverage is found, its fields come from the trusted setup.

Operation: consume a complete recipe

The inventory performs two loops. The first loop is read-only and rejects an unavailable recipe. The second loop executes only after every requirement is known to fit.

```

for each (ingredient, required) in recipe:
    if quantityOf(ingredient) < required: reject

for each (ingredient, required) in recipe:
    quantity[ingredient] = quantityOf(ingredient) - required
  
```

This mutation order is the main correctness proof. With water 40, coffee 18, and milk 100, a latte requiring 120 ml of milk fails during the first loop. Water and coffee remain untouched.

Operation: dispense the selected drink

`dispense()` checks selection and payment, then asks the inventory to consume the recipe. On success, it records the result and clears both session fields.

```

if selected is missing: reject
if balance < selected.price: reject

inventory.consume(selected.recipe)
result = DispenseResult(selected, balance, balance - selected.price)
clear selected beverage and balance
return result
  
```


`select()` gives early feedback about an unavailable drink. `consume()` owns the check that protects its stock mutation. `dispense()` delegates to it directly, without another availability check of its own. If consumption fails, the result and session reset are never reached, so the customer can still cancel for a refund.

`insertCoin()` checks that a drink is selected and the denomination is supported, then adds the amount. `cancel()` saves the current balance, clears the session, and returns the saved amount. Built-in exceptions are sufficient for the rejected operations.

Complete Runnable Implementation

The solution uses standard Java packages without a framework:

```
solution/
└─ src/
    └─ main/java/coffeevendingmachine/
        └─ model/
            ├── Beverage.java
            ├── DispenseResult.java
            ├── Ingredient.java
            └── IngredientInventory.java
        └─ service/
            └── CoffeeVendingMachine.java
        └─ demo/
            └── Main.java
    └─ test/java/coffeevendingmachine/
        └── SolutionTest.java
```

The `model` package owns domain values and ingredient state. `service` contains the public purchase workflow. `demo` and the test root depend on that API; the model never depends on the service.

What you would type in the round

The six application files under `src/main/java`, including the demo, contain 170 lines with imports and blank lines included. Of those, 136 lines define the model and workflow; 34 lines configure two drinks and run one purchase. This is the complete application reproduced in the appendix.

A possible 60-minute plan is 5 minutes for scope, 10 for class design, 30 to type the application and demo, 5 to verify, and 10 for questions or corrections. Typing speed varies; the budget is a planning estimate. The code has four customer operations and two stock loops, with routine records filling in the data model.

`SolutionTest.java` is additional study support. During the round, run the short demo and trace a failed purchase. The test file automates those checks so you can change the implementation while practicing.

Source files:

- `CoffeeVendingMachine.java` (solution/src/main/java/coffeevendingmachine/service/CoffeeVendingMachine.java)
- `IngredientInventory.java` (solution/src/main/java/coffeevendingmachine/model/IngredientInventory.java)
- `Beverage.java` (solution/src/main/java/coffeevendingmachine/model/Beverage.java)
- `DispenseResult.java` (solution/src/main/java/coffeevendingmachine/model/DispenseResult.java)
- `Ingredient.java` (solution/src/main/java/coffeevendingmachine/model/Ingredient.java)
- `Main.java` (solution/src/main/java/coffeevendingmachine/demo/Main.java)
- `SolutionTest.java` (solution/src/test/java/coffeevendingmachine/SolutionTest.java)

Run from `solution/`:

```
mkdir -p out
javac -d out $(find src/main/java src/test/java -name '*.java' | sort)
java -ea -cp out coffeevendingmachine.SolutionTest
java -cp out coffeevendingmachine.demo.Main
```

Verified with `javac 25.0.4.1`:

```
All coffee vending machine checks passed.  
Dispensed Latte; change: 50 cents; milk remaining: 380 ml
```

The matching PDF contains this same article followed by every source and test file in Appendix: Complete Runnable Code.

Verification Walkthrough

Start with a latte priced at 250 cents. Its recipe needs 40 ml water, 18 g coffee, and 120 ml milk. Inventory starts with 500 ml water, 100 g coffee, and 500 ml milk.

- 1 `select("LAT")` resolves the latte and asks `IngredientInventory.canPrepare(recipe)`. All quantities fit, so the machine stores the beverage. Inventory has not changed.
- 2 `insertCoin(200)` and `insertCoin(100)` move the session balance to 300 cents.
- 3 `dispense()` confirms that the 300-cent balance covers the 250-cent price and calls `IngredientInventory.consume(recipe)`.
- 4 `IngredientInventory.consume(recipe)` completes its validation pass, then decrements all three quantities. Water becomes 460, coffee becomes 82, and milk becomes 380.
- 5 The machine creates the latte result with 50 cents change, clears the selected beverage and balance, and returns the result.
- 6 An immediate second `dispense()` is rejected because no selection exists. Ingredient quantities remain at the values from the one successful drink.

For the rejection trace, try dispensing after inserting only 200 cents. The payment check fails before any stock mutation; adding another coin can complete the same purchase, or cancelling can refund it. Separately, a recipe that lacks milk must leave the available water and coffee untouched.

The study checks exercise caller errors, a successful retry with change, cancellation and session reset, and all-or-nothing stock consumption. They use valid setup data throughout.

Extensibility

Track physical change coins

Add a `ChangeInventory` that can both calculate and reserve a denomination combination. `dispense()` must verify drink ingredients and change availability before either resource changes. That creates a larger atomic operation and belongs beyond the junior base.

Add drink customizations

Represent a customer order separately from the configured `Beverage`. The order can derive an adjusted recipe and price for an extra shot or different size. The existing inventory API can still consume the resulting recipe map.

Model real brewing progress

Introduce states such as `HEATING`, `GRINDING`, `BREWING`, `DISPENSING`, and `FAILED` only when hardware reports asynchronous events. A controller would then advance legal transitions and decide how to refund after failure.

Support restocking

Add `restock(ingredient, amount)` to `IngredientInventory` and reject non-positive amounts at that new caller boundary. If administrators can edit recipes or prices too, validate those fields when accepting the edit. This changes the input contract; the base demo still uses fixed configuration.

What Is Expected at Each Level

Junior

A strong junior solution should identify the machine, beverage, and ingredient inventory; use integer money; keep quantities non-negative; validate the full recipe before decrementing anything; and demonstrate both a successful purchase and rejected operations. A map-based recipe and four public machine operations are enough. Strategy, State, Factory, and hardware classes are unnecessary for this contract.

An interviewer may hint that ingredient consumption needs two passes or that money should use integer cents. After that hint, the candidate should be able to finish the application and a short demonstration. More detailed automated tests can be useful during practice.

Mid-level

A mid-level candidate should reach the all-or-nothing recipe invariant without prompting, defend the source of truth for inventory, explain the trusted-setup boundary, and describe how change inventory would expand the transaction boundary.

Senior

A senior candidate should clarify concurrent sessions and physical failure semantics early. They should describe how to reserve ingredients and change atomically, how an asynchronous brewer reports completion, and which guarantees remain local versus distributed. Those concerns should stay out of the junior implementation unless the interviewer changes the contract.

Appendix: Complete Runnable Code

Every production source, test, and required build file from the verified solution is reproduced below. Paths are relative to the article directory.

Appendix manifest

APPENDIX_MANIFEST_BEGIN

```
path: solution/src/main/java/coffeevendingmachine/demo/Main.java
sha256: 1ae4a35c1aa88774446b77952dca70d677540d80e669d35282d6245b6201319f
path: solution/src/main/java/coffeevendingmachine/model/Beverage.java
sha256: 183b64c3bc632359c5806ba4246de442bba50770060bc80a952e80b9f8b4f9d1
path: solution/src/main/java/coffeevendingmachine/model/DispenseResult.java
sha256: 8e32f8067dd813dea2e59473d3262cfc791ba9d0885a9a09c9eea963d3cc7e92
path: solution/src/main/java/coffeevendingmachine/model/Ingredient.java
sha256: cc62973e8de1bb7ec4aa7c1f650bd55537a8fcc40274f40ad3833187ae667fb
path: solution/src/main/java/coffeevendingmachine/model/IngredientInventory.java
sha256: 92260d706e901b892c8f919df2f75c41e53ccf41acdd763a0021d179130a945c
path: solution/src/main/java/coffeevendingmachine/service/CoffeeVendingMachine.java
sha256: db460720937acc0eb9be0ffd88a2cf995848fe747b12cfebdc4b79d764b5e2b0
path: solution/src/test/java/coffeevendingmachine/SolutionTest.java
sha256: bc1cd1dd639a17de11631a0365764bbb1de19dd27a93fcd58383d609ff9a92c7
APPENDIX_MANIFEST_END
```

solution/src/main/java/coffeevendingmachine/demo/Main.java

```
package coffeevendingmachine.demo;

import coffeevendingmachine.model.Beverage;
import coffeevendingmachine.model.DispenseResult;
import coffeevendingmachine.model.Ingredient;
import coffeevendingmachine.model.IngredientInventory;
import coffeevendingmachine.service.CoffeeVendingMachine;
import java.util.List;
import java.util.Map;

public final class Main {
    public static void main(String[] args) {
        Beverage espresso = new Beverage("ESP", "Espresso", 150, Map.of(
            Ingredient.WATER_ML, 50, Ingredient.COFFEE_GRAMS, 18));
        Beverage latte = new Beverage("LAT", "Latte", 250, Map.of(
            Ingredient.WATER_ML, 40, Ingredient.COFFEE_GRAMS, 18,
            Ingredient.MILK_ML, 120));
        IngredientInventory inventory = new IngredientInventory(Map.of(
            Ingredient.WATER_ML, 500,
            Ingredient.COFFEE_GRAMS, 100,
            Ingredient.MILK_ML, 500));
        CoffeeVendingMachine machine = new CoffeeVendingMachine(
            List.of(espresso, latte), inventory);

        machine.select("LAT");
        machine.insertCoin(200);
        machine.insertCoin(100);
        DispenseResult result = machine.dispense();

        System.out.printf("Dispensed %s; change: %d cents; milk remaining: %d ml%n",
            result.beverage().name(), result.changeCents(),
            inventory.quantityOf(Ingredient.MILK_ML));
    }
}
```

solution/src/main/java/coffeevendingmachine/model/Beverage.java

```
package coffeevendingmachine.model;

import java.util.Map;

public record Beverage(
    String code,
    String name,
    int priceCents,
    Map<Ingredient, Integer> recipe) {}
```

solution/src/main/java/coffeevendingmachine/model/DispenseResult.java

```
package coffeevendingmachine.model;

public record DispenseResult(
    Beverage beverage,
    int paidCents,
    int changeCents) {}
```

[solution/src/main/java/coffeevendingmachine/model/Ingredient.java](#)

```
package coffeevendingmachine.model;

public enum Ingredient {
    WATER_ML,
    COFFEE_GRAMS,
    MILK_ML
}
```

[solution/src/main/java/coffeevendingmachine/model/IngredientInventory.java](#)

```
package coffeevendingmachine.model;

import java.util.EnumMap;
import java.util.Map;

public final class IngredientInventory {
    private final Map<Ingredient, Integer> quantities = new EnumMap<>(Ingredient.class);

    public IngredientInventory(Map<Ingredient, Integer> initialQuantities) {
        quantities.putAll(initialQuantities);
    }

    public boolean canPrepare(Map<Ingredient, Integer> recipe) {
        for (var entry : recipe.entrySet()) {
            if (quantityOf(entry.getKey()) < entry.getValue()) {
                return false;
            }
        }
        return true;
    }

    public void consume(Map<Ingredient, Integer> recipe) {
        if (!canPrepare(recipe)) {
            throw new IllegalStateException("Not enough ingredients for this beverage");
        }

        for (var entry : recipe.entrySet()) {
            Ingredient ingredient = entry.getKey();
            quantities.put(ingredient, quantityOf(ingredient) - entry.getValue());
        }
    }

    public int quantityOf(Ingredient ingredient) {
        return quantities.getOrDefault(ingredient, 0);
    }
}
```

solution/src/main/java/coffeevendingmachine/service/CoffeeVendingMachine.java

```

package coffeevendingmachine.service;

import coffeevendingmachine.model.Beverage;
import coffeevendingmachine.model.DispenseResult;
import coffeevendingmachine.model.IngredientInventory;
import java.util.Collection;
import java.util.HashMap;
import java.util.Map;
import java.util.Set;

public final class CoffeeVendingMachine {
    private static final Set<Integer> SUPPORTED_COINS = Set.of(25, 50, 100, 200);

    private final Map<String, Beverage> menuByCode = new HashMap<>();
    private final IngredientInventory inventory;
    private Beverage selected;
    private int balanceCents;

    public CoffeeVendingMachine(
        Collection<Beverage> menu,
        IngredientInventory inventory) {
        for (Beverage beverage : menu) {
            menuByCode.put(beverage.code(), beverage);
        }
        this.inventory = inventory;
    }

    public void select(String code) {
        if (selected != null) {
            throw new IllegalStateException("Cancel or dispense the current selection first");
        }

        Beverage beverage = menuByCode.get(code);
        if (beverage == null) {
            throw new IllegalArgumentException("Unknown beverage code: " + code);
        }
        if (!inventory.canPrepare(beverage.recipe())) {
            throw new IllegalStateException("Beverage is unavailable: " + code);
        }
        selected = beverage;
    }

    public void insertCoin(int cents) {
        if (selected == null) {
            throw new IllegalStateException("Select a beverage first");
        }
        if (!SUPPORTED_COINS.contains(cents)) {
            throw new IllegalArgumentException("Unsupported coin: " + cents);
        }
        balanceCents += cents;
    }

    public DispenseResult dispense() {
        if (selected == null) {
            throw new IllegalStateException("Select a beverage first");
        }
        if (balanceCents < selected.priceCents()) {
            throw new IllegalStateException("Insufficient payment");
        }

        inventory.consume(selected.recipe());
        DispenseResult result = new DispenseResult(
            selected, balanceCents, balanceCents - selected.priceCents());
        clearSession();
        return result;
    }

    public int cancel() {
        int refund = balanceCents;
        clearSession();
        return refund;
    }

    private void clearSession() {
        selected = null;
        balanceCents = 0;
    }
}

```

solution/src/test/java/coffeevendingmachine/SolutionTest.java

```

package coffeevendingmachine;

import coffeevendingmachine.model.Beverage;
import coffeevendingmachine.model.DispenseResult;
import coffeevendingmachine.model.Ingredient;
import coffeevendingmachine.model.IngredientInventory;
import coffeevendingmachine.service.CoffeeVendingMachine;
import java.util.EnumMap;
import java.util.List;
import java.util.Map;

public final class SolutionTest {
    public static void main(String[] args) {
        Beverage latte = new Beverage("LAT", "Latte", 250, Map.of(
            Ingredient.WATER_ML, 40, Ingredient.COFFEE_GRAMS, 18,
            Ingredient.MILK_ML, 120));
        IngredientInventory inventory = new IngredientInventory(Map.of(
            Ingredient.WATER_ML, 500, Ingredient.COFFEE_GRAMS, 100,
            Ingredient.MILK_ML, 500));
        CoffeeVendingMachine machine = new CoffeeVendingMachine(List.of(latte), inventory);

        // Caller errors must leave the purchase usable.
        expectRejected(() -> machine.select("UNKNOWN"));
        expectRejected(() -> machine.insertCoin(100));
        machine.select("LAT");
        expectRejected(() -> machine.select("LAT"));
        expectRejected(() -> machine.insertCoin(30));
        machine.insertCoin(200);
        expectRejected(machine::dispense);
        assert inventory.quantityOf(Ingredient.WATER_ML) == 500;
        assert inventory.quantityOf(Ingredient.COFFEE_GRAMS) == 100;
        assert inventory.quantityOf(Ingredient.MILK_ML) == 500;

        // The same session succeeds after adding enough money.
        machine.insertCoin(100);
        DispenseResult result = machine.dispense();
        assert result.beverage().equals(latte);
        assert result.paidCents() == 300;
        assert result.changeCents() == 50;
        assert inventory.quantityOf(Ingredient.WATER_ML) == 460;
        assert inventory.quantityOf(Ingredient.COFFEE_GRAMS) == 82;
        assert inventory.quantityOf(Ingredient.MILK_ML) == 380;
        expectRejected(machine::dispense);

        // Cancellation refunds only the current session and consumes nothing.
        machine.select("LAT");
        machine.insertCoin(100);
        assert machine.cancel() == 100;
        assert machine.cancel() == 0;
        assert inventory.quantityOf(Ingredient.WATER_ML) == 460;

        assert inventory.quantityOf(Ingredient.COFFEE_GRAMS) == 82;
        assert inventory.quantityOf(Ingredient.MILK_ML) == 380;
        machine.select("LAT");

        // Missing milk must not consume the water or coffee checked before it.
        Map<Ingredient, Integer> orderedRecipe = new EnumMap<>(Ingredient.class);
        orderedRecipe.putAll(latte.recipe());
        IngredientInventory shortStock = new IngredientInventory(Map.of(
            Ingredient.WATER_ML, 40, Ingredient.COFFEE_GRAMS, 18));
        expectRejected(() -> shortStock.consume(orderedRecipe));
        assert shortStock.quantityOf(Ingredient.WATER_ML) == 40;
        assert shortStock.quantityOf(Ingredient.COFFEE_GRAMS) == 18;
        assert shortStock.quantityOf(Ingredient.MILK_ML) == 0;
        CoffeeVendingMachine emptyMachine = new CoffeeVendingMachine(List.of(latte), shortStock);
        expectRejected(() -> emptyMachine.select("LAT"));

        System.out.println("All coffee vending machine checks passed.");
    }

    private static void expectRejected(Runnable action) {
        try {
            action.run();
        } catch (IllegalArgumentException | IllegalStateException expected) {
            return;
        }
        throw new AssertionError("Expected the operation to be rejected");
    }
}

```